

Simulazione 6

Esercizio 1: Puntatori e Memoria

Data la seguente porzione di programma, rispondi alle domande corrispondenti:

```
int main() {
    int* arr = new int[6] { 4, 1, 8, 3, 5, 2 };

    // 1. Le seguenti istruzioni sono corrette? Se si, cosa stampano?
    int a = arr[2];
    int& b = arr[2];
    a += 2;
    b += 3;
    cout << arr[2] << " " << a << endl;

    // 2. Le istruzioni seguenti sono corrette? Se si, cosa stampano?
    int* p = arr + 4;
    *(p - 1) = *p + arr[0];
    cout << arr[3] << endl;

    // 3. Perché la seguente sequenza genera un memory leak?
    int* temp = new int[3];
    temp = arr;

    // 4. Scrivere TUTTE le istruzioni per deallocare la memoria.
    return 0;
}
```

Esercizio 2: OOP (Gestione Prenotazioni Alberghiere)

Modellare una classe `GestioneHotel` per tenere traccia delle stanze e delle prenotazioni.

- Ogni stanza ha un numero (intero), un tipo (stringa, es. "Singola", "Doppia") e un costo a notte.
- La classe deve permettere di aggiungere nuove stanze al sistema.
- Fornire un metodo `prenota(string nome_cliente, string tipo_stanza)`: se c'è una stanza di quel tipo libera, la assegna al cliente e restituisce `true`. Se tutte le stanze di quel tipo sono occupate, il cliente viene messo in una coda d'attesa associata a quel tipo di stanza e il metodo restituisce `false`.
- Fornire un metodo `liberaStanza(int numero_stanza)`: libera la stanza. Se c'è qualcuno in coda d'attesa per quel tipo di stanza, la assegna automaticamente al primo in coda.

Esercizio 3: Ricorsione

Scrivere una funzione ricorsiva in C++ `int contaPari(int arr[], int size)` che, dato un array di interi e la sua dimensione, restituisca il numero di elementi pari presenti nell'array.

Simulazione 7

Esercizio 1: Puntatori e Memoria

Data la seguente porzione di programma, rispondi alle domande:

```
int main() {
    int* dati = new int[5] { 10, 20, 30, 40, 50 };

    // 1. Le seguenti istruzioni sono corrette? Se si, cosa stampano?
    int* mid = dati + 2;
    mid[1] = dati[0] + 5;
    cout << dati[3] << endl;

    // 2. Le istruzioni seguenti sono corrette? Se si, cosa stampano?
    int x = dati[4];
    int& y = x;
    y = 99;
    cout << dati[4] << " " << x << endl;

    // 3. Perché il seguente ciclo genera un errore/memory leak?
    while(*dati != 50) {
        cout << *dati << endl;
        dati++;
    }

    // 4. Scrivere le istruzioni per deallocare la memoria.
    return 0;
}
```

Esercizio 2: OOP (Sistema Ticket Assistenza)

Modellare una classe `SistemaTicket` per gestire l'assistenza clienti.

- Ogni ticket è identificato da un ID (stringa), un nome utente e una descrizione.
- Fornire un metodo `apriTicket(Ticket t)` che inserisce il ticket in una coda di elaborazione.
- Fornire un metodo `risolviProssimo()`: rimuove il ticket in testa alla coda, lo salva in uno storico (associando al nome utente il numero di ticket risolti per lui) e stampa un messaggio.
- Implementare una classe derivata `SistemaTicketPremium`. Sovrascrivere `apriTicket` in modo che se l'utente è "VIP" (verificato tramite una mappa di utenti VIP passata nel costruttore), il suo ticket salti la fila e venga elaborato per primo.

Esercizio 3: Liste Concatenate

Data la struttura `struct Nodo { int info; Nodo* next; };`, scrivere una funzione `int sommaNodi(Nodo* testa)` che calcoli iterativamente o ricorsivamente la somma di tutti i valori presenti nella lista.

Simulazione 8

Esercizio 1: Puntatori e Memoria

```
void f(int* p) {
    *(p + 1) = *p * 2;
}

int main() {
    int* val = new int[4] { 3, 5, 7, 9 };

    // 1. Le seguenti istruzioni sono corrette? Se si, cosa stampano?
    f(val + 1);
    cout << val[2] << endl;

    // 2. Le istruzioni seguenti sono corrette? Se si, cosa stampano?
    int& ref = val[3];
    ref = val[0] + val[1];
    cout << val[3] << endl;

    // 3. Spiegare perche' la seguente funzione causa un memory leak:
    // void elabora() { int* tmp = new int[10]; if(true) return; delete[] tmp; }

    // 4. Scrivere le istruzioni per deallocare 'val'.
    return 0;
}
```

Esercizio 2: OOP (Gestione Magazzino Elettronica)

Modellare una classe `Magazzino`.

- La classe memorizza un catalogo di prodotti (Nome e Quantità disponibile).
- Metodo `aggiungiProdotto(string nome, int qta)`: se il prodotto esiste, somma la quantità, altrimenti lo aggiunge al catalogo.
- Metodo `vendiProdotto(string nome, int qtaRichiesta)`: se c'è abbastanza disponibilità, scala la quantità e restituisce `true`. Se non basta, la richiesta (nome prodotto e quantità mancante) viene messa in una coda di "Ordini Pendenti" e restituisce `false`.
- Metodo `ristampaOrdiniPendenti()`: svuota la coda degli ordini pendenti stampandoli a video.

Esercizio 3: Ricorsione

Scrivere una funzione ricorsiva `bool isPalindroma(string s, int inizio, int fine)` che restituisca `true` se la stringa passata è palindroma, `false` altrimenti.

Simulazione 9

Esercizio 1: Puntatori e Memoria

```
int main() {
    int* v = new int[5] { 1, 2, 3, 4, 5 };

    // 1. Le seguenti istruzioni sono corrette? Se si, cosa stampano?
    for(int i = 0; i < 2; i++) {
        v[i] = *(v + 4 - i);
    }
    cout << v[0] << " " << v[1] << endl;

    // 2. Le istruzioni seguenti sono corrette? Se si, cosa stampano?
    int c = v[2];
    int& d = v[2];
    c = 10;
    d = 20;
    cout << c << " " << v[2] << endl;

    // 3. Memory Leak: Perché l'istruzione v = new int[3]; subito dopo
    // l'allocazione precedente genera un leak?

    // 4. Scrivere la deallocazione corretta.
    return 0;
}
```

Esercizio 2: OOP (Tracker Fitness)

Modellare una classe TrackerFitness.

- Memorizza gli esercizi svolti da un utente in varie date (es. mappa che associa una Data a una Lista/Vector di Esercizi). Ogni Esercizio ha Nome e CalorieBruciate.
- Metodo `aggiungiEsercizio(string data, Esercizio e)`: registra l'esercizio in quella data.
- Metodo `calorieTotali(string data)`: restituisce la somma delle calorie bruciate in quella specifica data.
- Implementare una classe derivata `TrackerPro`. Questa classe assegna un "Trofeo" (stringa salvata in un vector di trofei) ogni volta che l'utente supera le 1000 calorie totali in una singola data. Sovrascrivere i metodi necessari.

Esercizio 3: Liste Concatenate

Data la struttura `Nodo`, scrivere una funzione `void inserisciTesta(Nodo*& testa, int valore)` che allochi un nuovo nodo e lo inserisca in cima alla lista concatenata, aggiornando correttamente il puntatore alla testa.

Simulazione 10

Esercizio 1: Puntatori e Memoria

```
int main() {
    int* numeri = new int[4] { 7, 14, 21, 28 };

    // 1. Le seguenti istruzioni sono corrette? Se si, cosa stampano?
    int* p1 = numeri;
    int* p2 = &numeri[2];
    *p1 = *p2 + 2;
    cout << numeri[0] << endl;

    // 2. Le istruzioni seguenti sono corrette? Se si, cosa stampano?
    int& ref = numeri[1];
    int val = ref;
    ref++;
    cout << val << " " << numeri[1] << endl;

    // 3. Spiega il rischio dell'istruzione delete numeri; (senza le quadre [])

    // 4. Scrivere la deallocazione corretta.
    return 0;
}
```

Esercizio 2: OOP (Piattaforma Streaming)

Modellare una classe `PiattaformaStreaming`.

- Mantiene il catalogo dei Film (Titolo e Visualizzazioni totali).
- Mantiene una traccia dei film attualmente "In visione" da parte degli utenti (mappa che associa `NomeUtente` al Titolo del film).
- Metodo `iniziaVisione(string utente, string titolo)`: se il film è nel catalogo, lo segna come "In visione" per quell'utente. Se l'utente stava già guardando un altro film, l'operazione fallisce e restituisce `false`.
- Metodo `terminaVisione(string utente)`: l'utente finisce di guardare il film. Rimuove l'utente dalla mappa "In visione" e incrementa di 1 le visualizzazioni totali di quel film nel catalogo.

Esercizio 3: Ricorsione

Scrivere una funzione ricorsiva `int trovaMassimo(int arr[], int size)` che restituisca il valore massimo presente in un array di interi.